

## I<sup>2</sup>C Communication, Scheduling, and Interrupts on an ESP32

Vance Borus 2201308 , Sean Bubernak 2234086

5/8/2026

Assignment: Lab 3 Report

|                                    |          |
|------------------------------------|----------|
| <b>Introduction</b>                | <b>1</b> |
| <b>Methods and Techniques</b>      | <b>1</b> |
| <b>Experimental Results</b>        | <b>2</b> |
| Part 1                             | 2        |
| Part 2                             | 3        |
| Part 3                             | 4        |
| <b>Code Documentation</b>          | <b>5</b> |
| Part 1                             | 5        |
| Part 2                             | 5        |
| Part 3                             | 5        |
| <b>Overall Performance Summary</b> | <b>6</b> |
| <b>Teamwork Breakdown</b>          | <b>6</b> |
| <b>Discussion and Conclusions</b>  | <b>6</b> |
| <b>References</b>                  | <b>6</b> |
| <b>Appendix</b>                    | <b>7</b> |
| Total Number of hours spent: 10    | 7        |

## Introduction

The purpose of this lab is to explore task scheduling, interrupts, and communication over I2C. I2C is a two-wire protocol that allows serial communication of messages between the leader device and one or more follower devices. For this lab, we explore using I2C to send text to an LCD screen, first with the Arduino Wire library, and next with the LiquidCrystal library. We also explore task scheduling, implementing a full non-preemptive priority based scheduler, and demonstrate its capabilities with different priority levels. Finally we explore interrupts by creating three types of ISRs, external, timer-based, and BLE-based interrupts.

## Methods and Techniques

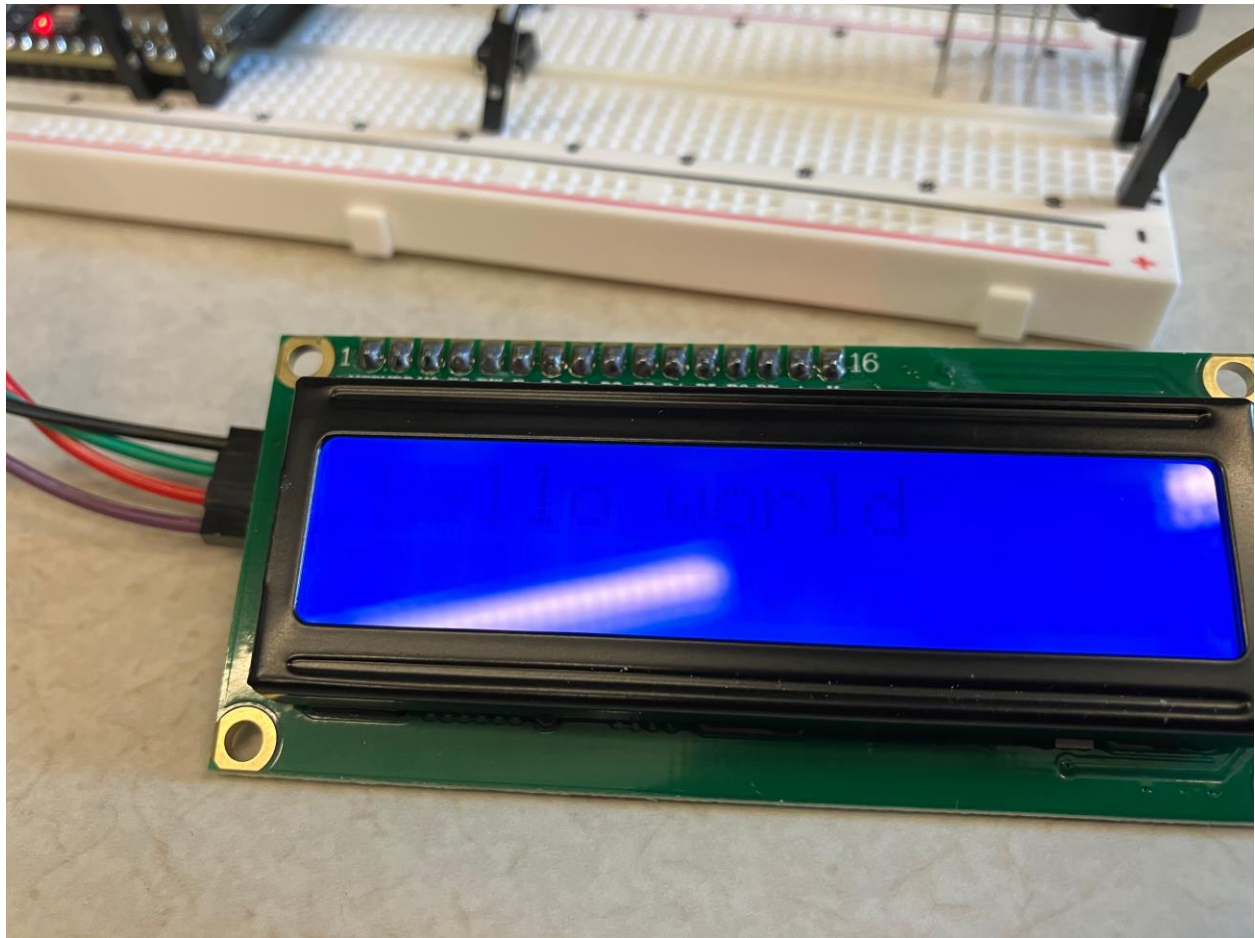
This lab included three primary topics. Those being, utilization of the LCD screen (and the serial monitor for that matter), scheduling tasks using a non-preemptive scheduling algorithm, and integrating interrupts and ISRs into those schedules of tasks. By strictly following the provided spec and designing with an AI last (if need be) we were able to prioritize learning the material which was beneficial as each task built on each other. An example of this would be the writeNibbleToLCD and writeCharacterToLCD we made in part 1. Although in parts 2 and 3 we used the lcd commands, without first understanding what was happening under the hood, using

the higher level commands used in the liquid crystal display library would have been less trivial to write and much more difficult to debug.

## Experimental Results

Describe specific test results which show how your code and hardware worked together to meet the lab requirements. Include 1-3 photos of your hardware setup.

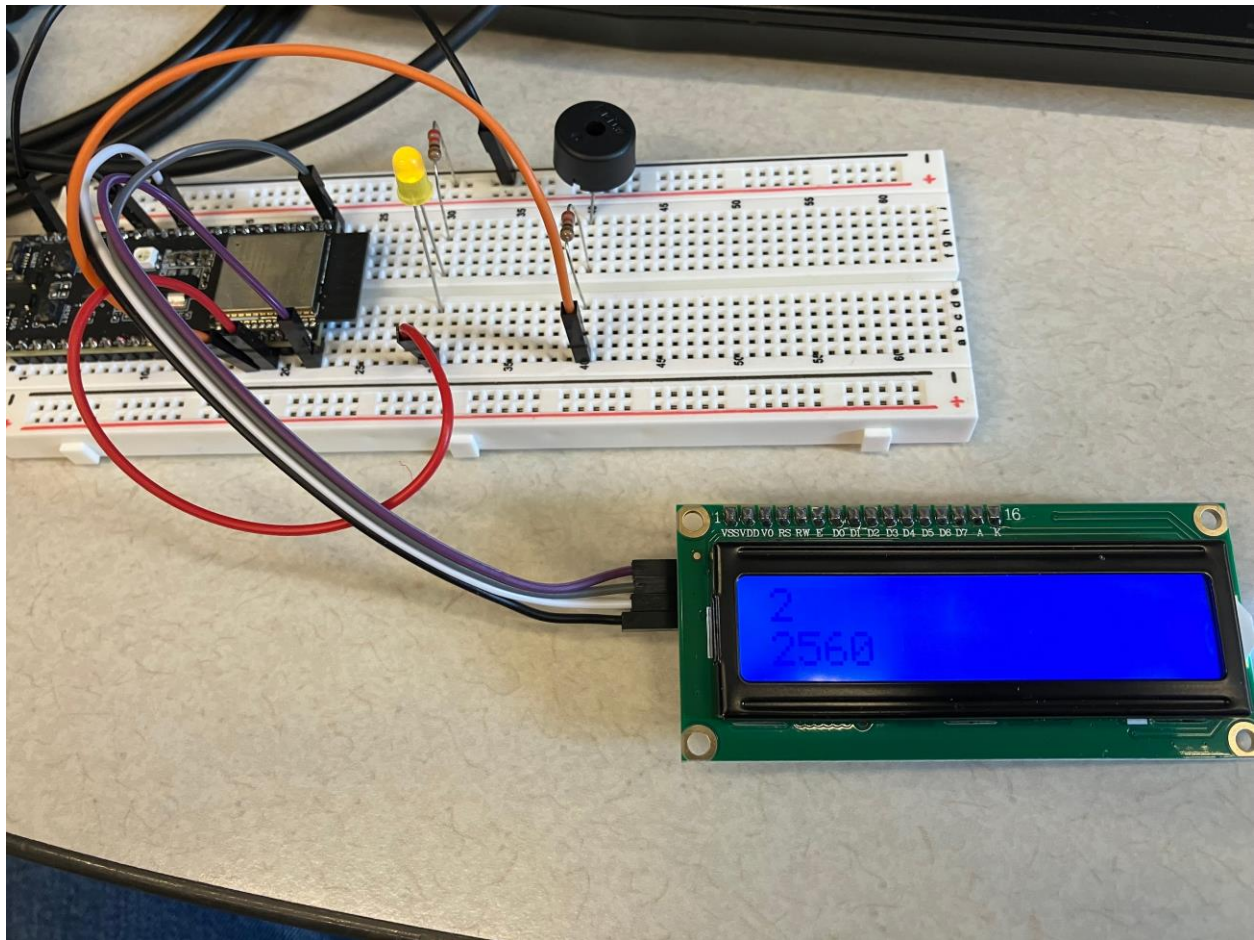
### Part 1



This part included implementing the ability to write text into the serial monitor and see it produced on the lcd screen. Here we typed, "hello world", into the monitor and as you can see in the above photo, it was replicated there on the screen. This demonstrates correct functionality of this part's specifications.

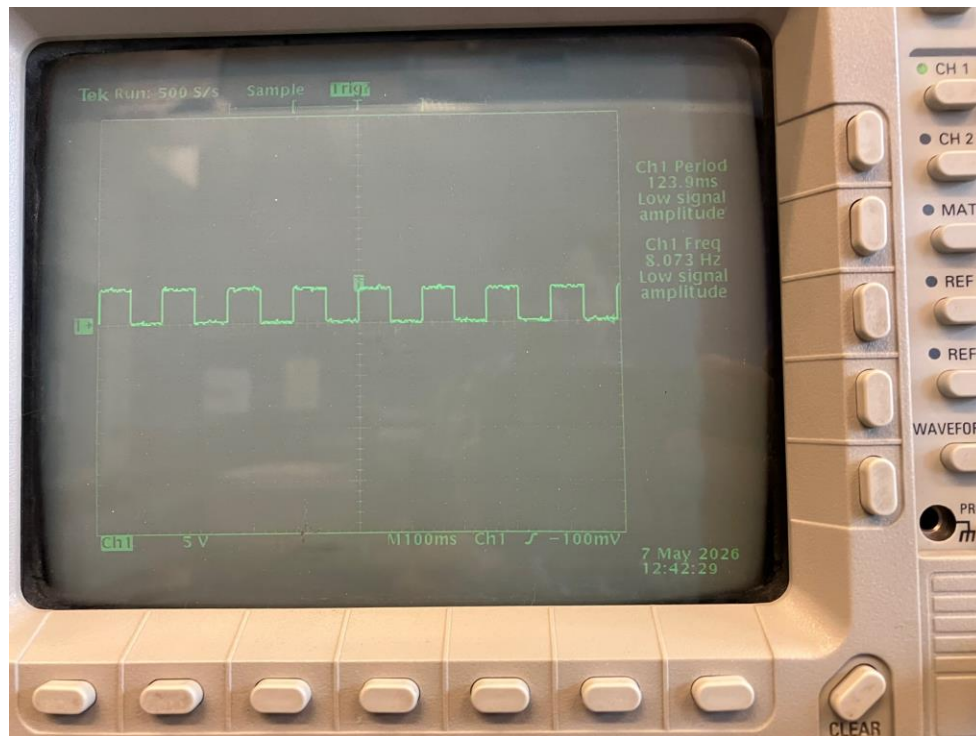
We used the lcd library only for initializing the lcd and wrote the rest of the logic solely using the lower-level, Wire library. However, by doing it this way we were able to learn how to interface with the wires the lcd monitor has and how the physical backpack module splits up our 1 byte characters (8 bits) into 4 bit nibbles.

## Part 2

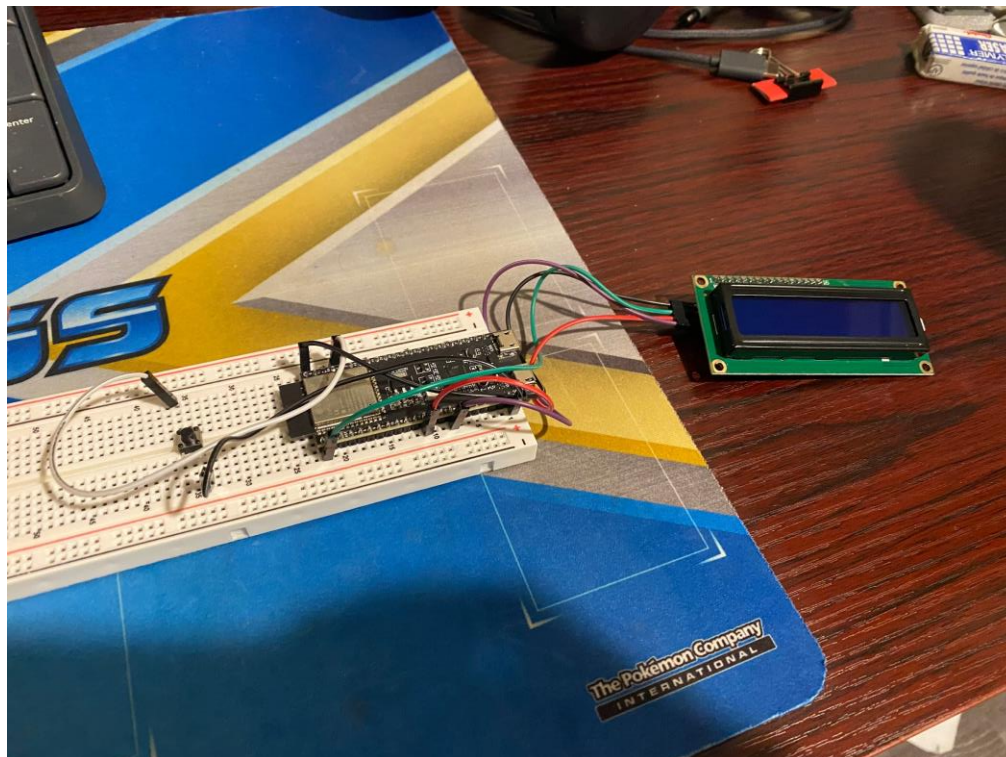


We were able to make our priority-based scheduler with 5 tasks. Here we can see the LED blink task, buzzer task, and counting task, with statuses shown on the LCD screen.

We found that all of the tasks appeared to run at the same time, indicating that our scheduler was properly executing these tasks fast enough to make them appear simultaneous. We also were able to hear the displayed frequencies on the buzzer, and could play short songs. We measured the frequency of the LED blink, and found that it correctly blinked at ~8 Hz.



### Part 3



This part of the lab built on the idea of the scheduler that we made in part two but integrates the idea of interrupts, specifically from separate sources, (both a pushbutton and the LightBlue app could trigger interrupts visualized on the lcd display. Ultimately, we were able to have the counter going in row 0 of the display and the notification of if a particular interrupt occurred in row 1 of the lcd. This is exactly how this part is described in the spec

## Code Documentation

Write a short paragraph for each task and each initialization process in your software. Identify the line number ranges for the tasks, function and struct names, etc in your paragraph. If some code is re-used from a previous lab, do not repeat its documentation. Reference the specific paragraph of your previous report.

### Part 1

This part served as an introduction to using the wire library to communicate with the lcd. There are different ways to go about doing this but we opted for using a writeNibbleToLCD function that we then used in our wrapper module, writeCharacterToLCD. To write just a nibble we mask in the bits we need to control (0b1101) and latch (0b1001). To write a full character we first divide up a uint8 into 2 nibbles, a top and bottom, and then write to each using the write nibble function. We are able to parse inputs by running a for loop in void loop that loops until the serial monitor encounters an endline character. Each character that is encountered is written to the LCD using the above mentioned write character function.

### Part 2

For the task struct, we kept the basic template, but also allowed each task to know its own period, simplifying the process of checking when a task should move from SLEEPING -> READY.

### Part 3

In part 3, when making our ISRs for each interrupt, we had to ensure that they were very short so as to avoid breaking other parts of the code. We accomplished this by introducing flag variables at the top of the file. These flags would then be checked by the main loop() function periodically to print the actual interrupt messages to the screen, avoiding overhead in the ISRs.

## Overall Performance Summary

In our demo session we were able to demonstrate perfect functionality (according to the provided spec) for each of the three tasks in this lab. We even took it a step further to show the waveform of the blinking LED in part two on the oscilloscope. Not only did we demonstrate functionality but we took it a step further to answer the why. Our demo served as a code review session as well. It was also very helpful to have the TA there to answer any questions or expand on any ideas we had.

## Teamwork Breakdown

We worked on all three parts of the lab in parallel, using git, then merging our parts together into one file for submission. For part 1, we each implemented it separately so we got up to speed on how I2C communication worked. For part 2, Vance wrote the scheduler while Sean wrote the tasks. For part 3 Vance did the buttons and timer interrupts while Sean worked on setting up bluetooth and its callback interrupt.

## Discussion and Conclusions

Overall the most challenging part of this lab for the two of us was part two. Not only was it the most involved, (a scheduler with configurable priorities as well as six different tasks to handle), but despite going through scheduling on paper in class several times, this was our first experience with actually coding one (we both did ICTE 5 after this and due to this order, it was trivial). Getting this right for the first time in task two made integrating interrupts into a more basic schedule much easier than we had initially anticipated. Looking back on the earlier lab where we had to control the blink frequency of two independent LEDs, if we had instead handled this with a schedule it would have been much simpler to implement. This is a very important skill to develop and understand the workings of under the hood.

## References

[1] PlatformIO, “Quick Start — PlatformIO Core (CLI) Documentation,” *PlatformIO Documentation*, 2014–present. Accessed: Apr. 21, 2026. [Online]. Available: <https://docs.platformio.org/en/latest/core/quickstart.html>

[2] Espressif Systems, *ESP32-S3-DevKitC-1 User Guide*. Espressif Documentation, 2016–2026. Accessed: Apr. 21, 2026. [Online]. Available: <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html>

# Appendix

Total Number of hours spent: 10